ORBITA IQ

Technical Architecture & Project Status Assessment Report

1. Executive Summary

This report provides a comprehensive architectural and project status audit of the **ORBITA IQ** repository (`satellite-ops-auth`). The project aims to be an AI-Powered Satellite Conjunction Risk Intelligence Platform. Currently, the repository consists of a production-ready authentication module, a functional frontend shell (Mission Control Dashboard) powered by mock data, and an evolving backend architecture built on FastAPI. While the foundational infrastructure and security layers are well-implemented, the core scientific services (SGP4 propagation, TLE/OMM processing, SatGuard screening) and real-time data pipelines are still pending integration or full implementation.

---

2. Repository Overview

The repository is organized into three primary domains:

- **Frontend (`/frontend`)**: React + TypeScript + Vite application. Contains components, pages, hooks, and routing logic for the Mission Control Dashboard.
- **Backend (`/backend`)**: FastAPI application in Python. Structured using a service-layer pattern with routers, models, schemas, and services.
- **Database / Infra (`/supabase`)**: Contains SQL migrations for Supabase (PostgreSQL), defining the schema, Row Level Security (RLS) policies, and database functions.
- **Deployment**: Configured via `render.yaml` for the backend and `vercel.json` for the frontend.

---

3. Architecture Overview

Frontend Architecture

- **Framework**: React 18 with TypeScript, bundled via Vite.
- **State Management**: React Context API (`AuthContext.tsx`) and custom hooks (`useSatellites.ts`, `useAlerts.ts`, `useDashboard.ts`).
- **Routing**: `react-router-dom` with protected route wrappers (`<ProtectedRoute>`).
- **Component Structure**: Highly modularized (`src/components/` split into `auth`, `dashboard`, `orbit`, `satellites`, `alerts`, `layout`, `ui`).
- **UI Libraries**: Tailwind CSS with `shadcn/ui` components. CesiumJS for 3D globe visualization.

Backend Architecture

- **Framework**: FastAPI (Python 3.10+).
- **API Structure**: Versioned routing (`/api/v1/`) with domain-driven endpoints (`auth`, `satellites`, `alerts`, `orbit`, `cdm`, `omm`).
- **Service Layer**: Business logic decoupled from routes (e.g., `auth_service.py`, `satellite_service.py`, `conjunction_service.py`).

- **Repository Layer**: SQLAlchemy AsyncSession used directly within services.
- **Models/Schemas**: Pydantic for validation (`schemas/`), SQLAlchemy for ORM (`models/`).

Database Architecture
- **Platform**: Supabase (PostgreSQL).
- **Core Tables**: `profiles`, `login_audit_log`, `satellites`, `tle_records`, `omm_records`, `conjunction_events`, `cdm_records`, `orbit_state`.
- **Security**: Strict Row Level Security (RLS) ensuring operators and admins are compartmentalized.

Authentication Architecture
- **Provider**: Supabase Auth (GoTrue).
- **Login Flow**: Employee ID + Password. Backend resolves Employee ID to email via service role and delegates password validation to Supabase.
- **Session Handling**: JWT access and refresh tokens handled by `@supabase/supabase-js` client-side, with stateless server-side validation.
- **RBAC**: Defined in `profiles.role` (`admin` vs `operator`). Re-verified via `require_role()` dependency on every protected endpoint.

---

4. Database Audit
Current Implementation Status against Project Requirements:

Requirement	Actual Table Name	Status	Notes
`users`	`auth.users` / `profiles`	☑ Implemented	Supabase native auth mapped to `profiles` via triggers.
`satellites`	`satellites`	☑ Implemented	Exists in `0002_satellites_schema.sql`.
`orbit_state`	`orbit_state`	☑ Implemented	Exists in `0005_orbit_state_schema.sql`.
`alerts`	*None*	❌ Missing	Not found in migration files.
`alert_history`	*None*	❌ Missing	Not found in migration files.

Other Existing Tables: - `tle_records` (Foreign keys to `satellites`) - `omm_records` (Foreign keys to `satellites`) - `cdm_records` (Foreign keys to `satellites`) - `conjunction_events` (Tracks primary/secondary satellite risks) - `login_audit_log` (Authentication tracking)

Database Health & Normalization: - **Foreign Keys**: Correctly implemented with `ON DELETE CASCADE` where applicable. - **Indexes**: Time-series indexes created (e.g., `idx_tle_records_satellite_id_epoch`). - **Constraints**: Enums used properly (`satellite_status`, `object_type`, `risk_level`).

---

5. API Audit

Method	Endpoint	Purpose	Auth Required	Status
POST	`/api/v1/auth/login`	Authenticate user	No	☑ Implemented
POST	`/api/v1/auth/refresh`	Refresh session	No	☑ Implemented
POST	`/api/v1/auth/logout`	Terminate session	Yes	☑ Implemented
GET	`/api/v1/auth/me`	Fetch profile	Yes	☑ Implemented
POST	`/api/v1/auth/password-reset/request`	Password reset link	No	☑ Implemented
GET	`/api/v1/satellites`	List all satellites	Yes	Partially Implemented (Mocks)
GET	`/api/v1/satellites/{id}`	Get specific satellite	Yes	☑ Implemented
POST	`/api/v1/satellites/norad`	Add via NORAD ID	Yes	☑ Implemented
PUT	`/api/v1/satellites/{id}`	Update satellite metadata	Yes	☑ Implemented
DELETE	`/api/v1/satellites/{id}`	Delete satellite	Yes (Admin)	☑ Implemented
POST	`/api/v1/satellites/upload-tle`	Upload TLE	Yes	☑ Implemented
POST	`/api/v1/satellites/upload-omm`	Upload OMM JSON	Yes	☑ Implemented
GET	`/api/v1/alerts`	List conjunction alerts	Yes	Partially Implemented (Mocks)
PUT	`/api/v1/alerts/{id}/status`	Update alert state	Yes	☑ Implemented
POST	`/api/v1/alerts/seed`	Generate mock alerts	Yes	☑ Implemented
POST	`/api/v1/cdm/upload`	Upload CDM payload	Yes	Partially Implemented
GET	`/api/v1/dashboard`	Dashboard KPIs	Yes	Partially Implemented
GET	`/api/v1/orbit/{satellite_id}`	Fetch propagation data	Yes	Partially Implemented

Findings: - **Missing Endpoints**: Administrative User Management CRUD (`/users`), Conjunction Probability compute triggers, System Health/Logs. - **Broken Endpoints**: None identified, but several endpoints return hardcoded UUIDs and mock datasets.

---

6. Frontend Audit

Page / Screen	Implemented?	Notes
Dashboard (`DashboardPage.tsx`)	Partial	UI is complete. Fed by mock data hooks.
Satellites (`MySatellitesPage.tsx`)	Partial	UI is complete. Table and dialogs work via mock data.
Alerts (`AlertsPage.tsx`)	Partial	UI is complete. Filtering works on mock payload.
Orbit Monitoring (`OrbitViewerPage.tsx`)	Partial	Cesium globe renders, but propagation relies on simplified calculations, not real SGP4.
Admin	❌ Missing	No dedicated Admin Dashboard for user management exists.
Settings (`SettingsPage.tsx`)	☑ Implemented	Profile, Security, and Notification tabs exist.
Authentication Flow	☑ Implemented	Login, Forgot/Reset Password fully operational.

---

7. Security Audit

- **Authentication**: **Low Risk**. Supabase GoTrue handles heavy lifting. Passwords are never seen by the backend logic.
- **Authorization**: **Low Risk**. RLS policies enforce access. FastAPI `get_current_user` ensures stateless JWT validation.
- **SQL Injection**: **Low Risk**. SQLAlchemy ORM safely parameterizes all queries.
- **API Exposure**: **Medium Risk**. Endpoints are protected, but rate limits (`app.core.limiter`) only explicitly secure `/auth/login` and `/auth/password-reset/request`.
- **Environment Variables**: **Low Risk**. Secrets are excluded from version control (`.env.example` verified).
- **CORS**: Correctly configured to strictly allow the Vercel frontend URL.
- **Audit Logging**: Implemented via `login_audit_log` with lockout after consecutive failed attempts.

---

8. Performance Audit

- **Database Queries**: SQLAlchemy `selectinload` is used to prevent N+1 issues when fetching relationships (e.g., `Satellite` with `OrbitState`).
- **API Calls**: The dashboard currently fetches mocks. Real-time websocket streaming is missing for alerts and telemetry.
- **Frontend Rendering**: `CesiumJS` rendering is heavy. Without WebWorkers for SGP4 propagation, the UI thread will stall with large catalogs.
- **Caching**: No Redis caching layer detected for TLE fetching or heavy orbital calculations.

Current Capacity Estimates (Browser Performance w/ Cesium): - **5 Satellites**: Excellent. - **20 Satellites**: Good. - **50 Satellites**: Moderate (Simplified propagation runs on the UI thread). - **100+ Satellites**: Severe bottleneck expected unless calculation is moved to backend or WebWorkers.

---

9. Deployment Audit

- **Backend Config**: `render.yaml` exists and correctly maps the environment variables and start commands (`uvicorn app.main:app --host 0.0.0.0 --port $PORT`).
- **Frontend Config**: `vercel.json` exists, configuring rewrite rules suitable for a React SPA.
- **Readiness**:
  - Backend is Ready for deployment on Render.
  - Frontend is Ready for deployment on Vercel.
  - Supabase requires manual application of `.sql` migrations in the `/supabase/migrations` directory.

---

10. Feature Completion Matrix

Feature	Status	Completion %
Authentication	COMPLETED	100%
Admin Management	NOT IMPLEMENTED	0%
Employee Management	NOT IMPLEMENTED	0%
Satellite Registry	PARTIALLY COMPLETED	70%
NORAD Import	PARTIALLY COMPLETED	80%
TLE/OMM/CDM Import	PARTIALLY COMPLETED	60%
Orbit Monitoring	PARTIALLY COMPLETED	40% (Mocked)
Alert System	PARTIALLY COMPLETED	30% (Mocked)
Conjunction Screening	NOT IMPLEMENTED	0%
Risk Analysis & Timeline	NOT IMPLEMENTED	0%
Cesium Visualization	PARTIALLY COMPLETED	50%
Digital Twin	NOT IMPLEMENTED	0%
Mission Impact Analysis	NOT IMPLEMENTED	0%

Overall Project Completion: ~35%

---

11. Gap Analysis

Comparing the current implementation against the target **ORBITA IQ AI-Powered Platform**:

Critical Gaps: - Missing `alerts` and `alert_history` SQL tables. - Lack of a real SGP4 orbital propagation engine on the backend.

- Conjunction Risk probability algorithms (e.g., Alfano, Foster) are

undefined. - Missing live data ingestion pipelines (websockets) for continuous telemetry.

Architecture Gaps: - Lack of a message broker (RabbitMQ/Redis) or task queue (Celery) to handle heavy astronomical computations asynchronously. - Missing an Admin Dashboard for managing operator access and global system health.

---

12. Risk Assessment

1. **Scalability Risk (High):** Executing orbital math on the frontend (even simplified) will not scale to hundreds of objects. Needs to migrate to backend pre-computation.
2. **Technical Debt (Medium):** The frontend relies heavily on `src/mock/` data. Swapping this out for live API hooks will require significant refactoring of `useSatellites.ts` and `useAlerts.ts`.
3. **Database Constraints (Low):** The database schema is well thought out, though missing a few essential alert tracking tables.

---

13. Recommended Next Steps (Roadmap)

Phase 1: Backend Data Integrity & Real API Integration 1. Generate migrations for `alerts` and `alert_history` tables. 2. Implement CRUD endpoints for Administrative User Management. 3. Replace frontend mock data hooks with actual Axios/Fetch calls to the FastAPI backend.

Phase 2: Core Orbital Mechanics 1. Implement `sgp4_service.py` to handle propagation on the backend. 2. Build an async task queue to periodically fetch and update TLE data from CelesTrak. 3. Connect the Cesium globe to backend-propagated state vectors via WebSocket.

Phase 3: AI & Conjunction Intelligence 1. Implement the Conjunction Data Message (CDM) screening engine. 2. Integrate Machine Learning models for collision probability. 3. Build the Mission Impact Analysis dashboard.

---

14. Production Readiness Score

- **Demo Ready**: **Yes** (UI is presentable using mock data)
- **MVP Ready**: **No** (Core tracking/alert features are not yet connected to live logic)
- **Production Ready**: **No**

Overall Production Score: 25 / 100 *(Strong foundations in Auth, Security, and UI, but missing core business logic execution).*